Software Testing, Verification, and Validation (M.EIC)
Software Testing (MESW)

Static Testing

Ana Paiva

apaiva@fe.up.pt **www.fe.up.pt/~apaiva**

Agenda

- Software Quality Metrics
- Static testing

Theory of Measurement

- Metric = measurable characteristic of some entity
  - Primitive/Primary and derived metrics
  - Proxy metric for difficult-to-measure entities
- Values
  - Scales: nominal / ordinal / interval / ratio / proportional / percentage…
  - Expected (min / max / average), actual (ditto, trend)
- Validity (accuracy) of measurement
- Reliability (precision) of measurement

Types of Metrics in Software Engineering

- Product metrics
  - Characteristics of “the material” and (intermediate) results
- Process metrics
  - Characteristics of the work organization

Some Software Product Metrics

- Software size
  - Estimate product size
    - Function and Use case points
    - Story points
  - Measure size
    - Lines of Code (e.g., lines of Code (LOC) / Source Lines of Code (SLOC) / Delivered Source Lines of Code DSLOC / Total Lines of Code TLOC

    and…
    - Comments, blanks
    - Block delimiters, multiple statements on one line
    - … (Caution! – depend on programming language and formatting)
- Software Quality
  - Test coverage, defect density, and detection rate
  - Code quality and complexity
  - Product reliability, usability

Software Availability

- (A) Availability: the fraction of total time a system is operational and ready for use
- (MTBF) Mean Time Between Failures: The average time the system runs continuously before breaking down. This is the pure metric for Reliability.
- (MTTR) Mean Time To Repair: The average time required to diagnose, fix, and restart the system after a breakdown. This is the pure metric for Maintainability

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

HOW TO MEASURE THE QUALITY OF THE TESTS?

Test adequacy/coverage criteria

Adequacy/coverage criteria

- **Criteria to decide** if a given test suite **is adequate**, i.e., to give us “enough” confidence that “most” of the defects are revealed
  - Used in the evaluation and in the design/selection of test cases
  - In practice, reduced to coverage criteria

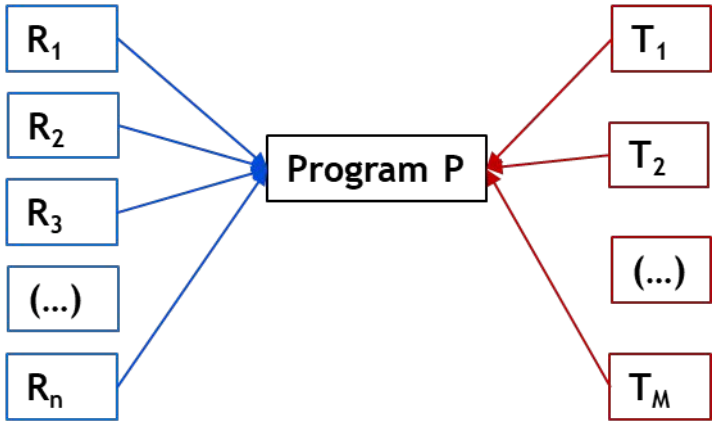


Coverage criteria

Consider a program ***P*** written to meet a set of ***R*** functional requirements, and a set ***T*** of tests to test ***P***

- Let *R* contain *n* requirements labeled R_1, R_2,…, R_n .
- A set *T* containing *M* tests constructed to test *P*.

P has been executed against each test in *T* and has produced correct behavior.



We now ask:

Is *T* good enough?

Has *P* been tested thoroughly?

Is *T* adequate?

Coverage criteria

- **Black box (depends solely on requirements for program P under test)**
  - Requirements/specification coverage (black-box)
    - At least one test case for each requirement/specification statement
  - Model coverage
    - State-transition coverage
    - Use case and scenario coverage

- **White box (depends solely on program P under test)**
  - Code coverage (white-box)
    - Control flow coverage (statement, decision, MC/DC coverage …)
    - Data flow coverage
  - Fault coverage

Measurement of adequacy example

- Consider that the program *sumProduct* must meet the following requirements:
  - **R1** - Input two integers, x and y, from the standard input device
  - **R2** - if x<y, print (z + y)
  - **R3** - if x≥ y, print (z x y).
- Consider criterion **C**: “all requirements coverage”
- Consider test **T** with test input data {<x=2, y=3>}

Is **T** adequate according to criterion **C**?

Measurement of adequacy example

- Consider that the program *sumProduct* must meet the following requirements:
  - **R1** - Input two integers, x and y, from the standard input device
  - **R2** - if x<y, print (z + y)
  - **R3** - if x≥ y, print (z x y).
- Consider criterion **C**: "all requirements coverage"
- Consider test **T** with test input data {<x=2, y=3>}

Is **T** adequate according to criterion **C**?

- T tests R1 and R2, but not R3,
- So, T is not adequate (*does not achieve 100% coverage according to criterion C*)
- T covers 2/3 requirements. T is 66% good according to C.

Agenda

- Software Quality Metrics
- Static testing

STATIC TESTING

Evaluates code, design documents, and other software artifacts without executing the code

Static testing

Can be performed well before dynamic test execution

Often much cheaper

Finds faults (cause of the failures)

Early defect detection and warnings about suspicious aspects

Improves development productivity

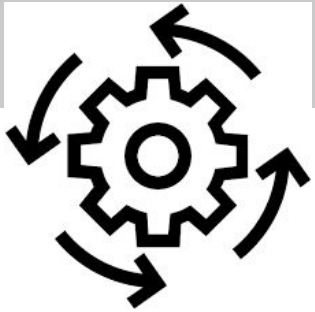
Reduces testing cost and time

STATIC TESTING TECHNIQUES RELY ON

Manual examination
REVIEWS



Automated analysis
STATIC ANALYSIS



REVIEWS



Reviews

Typical defects easier to find in reviews than in dynamic testing are:

- deviations from standards
- requirement defects
- design defects
- insufficient maintainability
- incorrect interface specifications

What can be reviewed/inspected?

- Policy, strategy, business plans, marketing materials, contracts
- System requirements, feasibility studies
- Test plans, test designs, test cases, test results
- System designs, logical & physical
- Software code
- User manuals, procedures, and training material

Reviews

Cost-effective

- Reduce faults by a factor of 10[1]
- 25% reduction in schedules, remove 80% - 95% of faults at each stage, 28 times reduction in maintenance cost, and many others[2]

Cost of Reviews

- Rough guide: 5%-15% of development effort (half a day a week is 10%)

[1] [Freedman & Weinberg, Handbook of Walkthroughs, Inspections & Technical Reviews]
[1] [Yourdon, Structured Walkthroughs]
[2] [Gilb & Graham, Software Inspection]

Reviews deliverable



CHANGES (EDITS) IN THE REVIEW PRODUCT



CHANGE REQUESTS



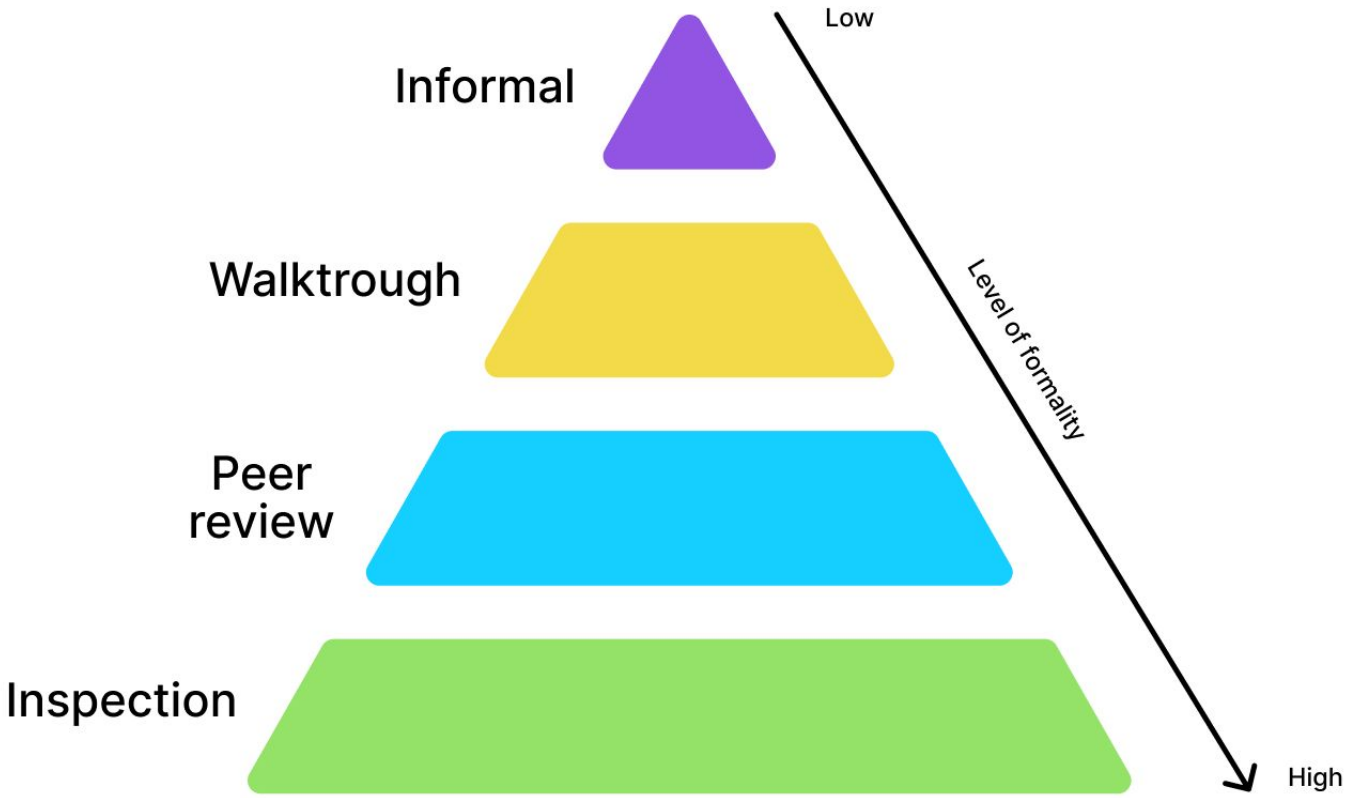
PROCESS IMPROVEMENT SUGGESTIONS



METRICS

Reviews

May vary in formality



Phases of a formal review



Individual preparation



Kick-off



Planning

Review meeting



rework



Follow-up

Phases of a formal review

Planning: selecting participants, allocating roles, defining entry and exit criteria for more formal review types (e.g., inspections), and determining which parts of documents to examine.

Kick-off: distributing documents; explaining the objectives, process and documents to the participants; and checking entry criteria (for more formal review types).

Individual preparation: work done by each of the participants on their own before the review meeting, noting potential defects, questions and comments.

Review meeting: discussion or logging, with documented results or minutes (for more formal reviews). Meeting participants may simply note defects, make recommendations, or make decisions regarding the defects.

Rework: fixing defects found, typically done by the author.

Follow-up: checking that defects have been addressed, gathering metrics and checking on exit criteria (for more formal review types).

Roles and Responsibilities

Manager: decides on the execution of reviews, allocates time in project schedules and determines if the review objectives have been met.

Moderator: the person who leads the review of the document or set of documents, including planning the review, running the meeting, and follow-up after the meeting. If necessary, the moderator may mediate between the various points of view and is often the person upon whom the success of the review rests.

Author: the writer or person with chief responsibility for the document(s) to be reviewed.

Reviewers: individuals with a specific technical or business background (also called checkers or inspectors) who, after the necessary preparation, identify and describe findings (e.g., defects) in the product under review. Reviewers should be chosen to represent different perspectives and roles in the review process and should take part in any review meetings.

Scribe (or recorder): documents all the issues, problems and open points that were identified during the meeting.

Success factors for reviews

Clear predefined objective

Involve the right people for the review objectives

Defects found are welcomed and expressed objectively

People issues and psychological aspects are dealt with

Apply review techniques appropriate to the software artifacts and reviewers

Use **checklists or roles**, where appropriate, to improve defect detection

Provide **training** in review techniques, especially for more formal methods, such as inspections

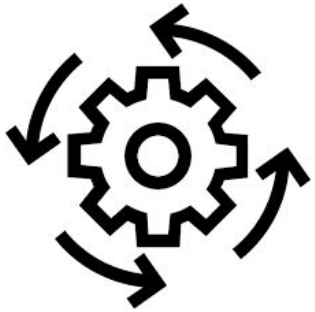
Management supports the review process by allocating time in project schedules.

Emphasis on learning and process improvement.

Reviews

Aspect	Informal Review	Walkthrough	Peer Review	Inspection
Purpose	Quick feedback and initial checks	To explain the work and gather suggestions	To detect defects and improve quality	To rigorously detect defects and ensure standards
Formality	Very low	Low to moderate	Moderate	High
Participants	Author + colleagues	Author + colleagues	Author + Peers (usually skilled colleagues)	Trained moderators, authors, peers/reviewers, and scribes
Preparation	Little or none	Some preparation by the author (led by the author)	Reviewers prepare before meeting	Thorough preparation by reviewers and author
Process	Informal discussion or ad-hoc review	Step-by-step presentation of the work	Structured review process with assigned roles	Highly structured process with defined roles and checklists
Focus	General understanding and quick fixes	Understanding and feedback	Finding defects	Detecting defects, adherence to standards, and metrics
Documentation	Minimal	Notes and comments	Review report or comments	Detailed review record, defect logs, and metrics
Outcome	Suggestions or minor fixes	Suggested improvements	Defect list and recommendations	Comprehensive defect list, decision on work product acceptance
Time Required	Short	Moderate	Moderate	Long, intensive

STATIC ANALYSIS



Static analysis

Static analysis tools are **typically used**

- **by developers** (checking against predefined rules or programming standards) before and during component and integration testing, and
- **by designers** during software modelling

Static analysis tools **may produce many warning messages**, which need to be well managed to allow the most effective use of the tool

Examples of tools:

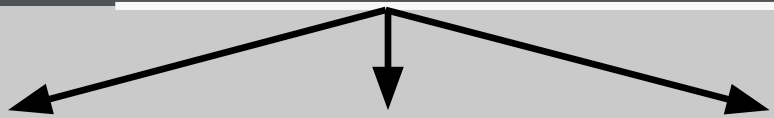
- SonarQube; PMD; Checkstyle; FindBugs; ESLint; Flake8
- Compilers may offer some support for static analysis, including the calculation of **metrics**

Static analysis

Typical defects discovered by static analysis tools include:

- referencing a variable with an undefined value
- inconsistent interface between modules and components
- variables that are never used
- unreachable (dead) code
- programming standards violations
- security vulnerabilities
- syntax violations of code and software models

STATIC ANALYSIS MAY BE PERFORMED THROUGH



CONTROL FLOW ANALYSIS

DATA FLOW ANALYSIS

(...)

Control flow analysis

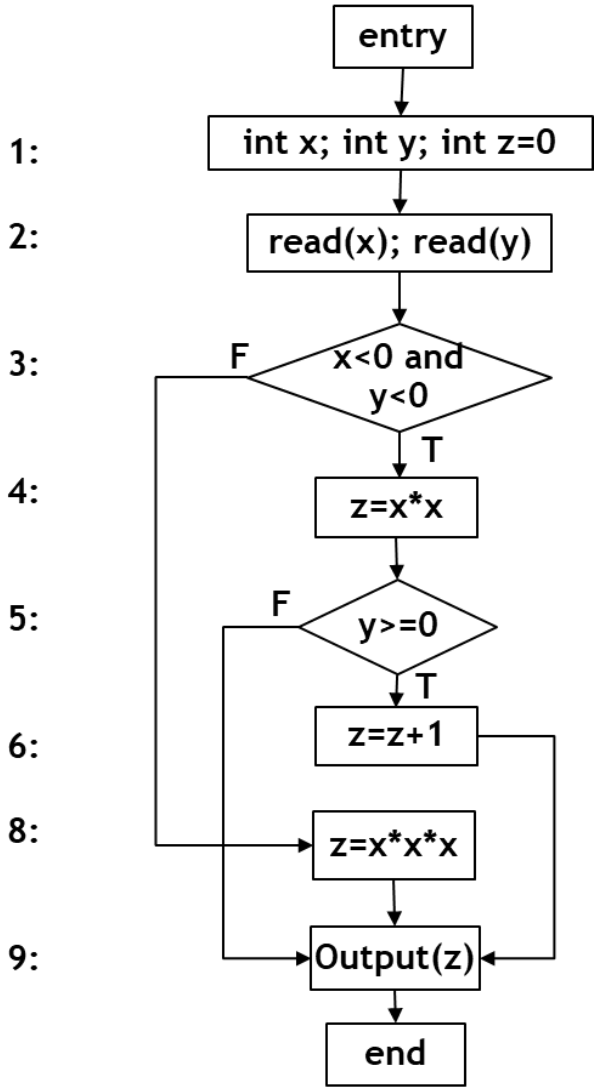
- Control analysis may find:
  - Nodes not accessible from the start node
  - Infinite loops
  - Multiple entries to loops
  - Whether code is well structured, i.e., reducible
  - Any jumps to undefined labels
  - Any labels not jumped to
  - Cyclomatic complexity and other metrics
  - …

Control flow analysis

```
1:  int x; int y; int z=0;
2:  read(x); read(y);
3:  if (x<0 and y<0) {
4:      z=x*x;
5:      if (y>=0)
6:          z=z+1;
7:  }
8:  else z=x*x*x;
9:  output(z);
```

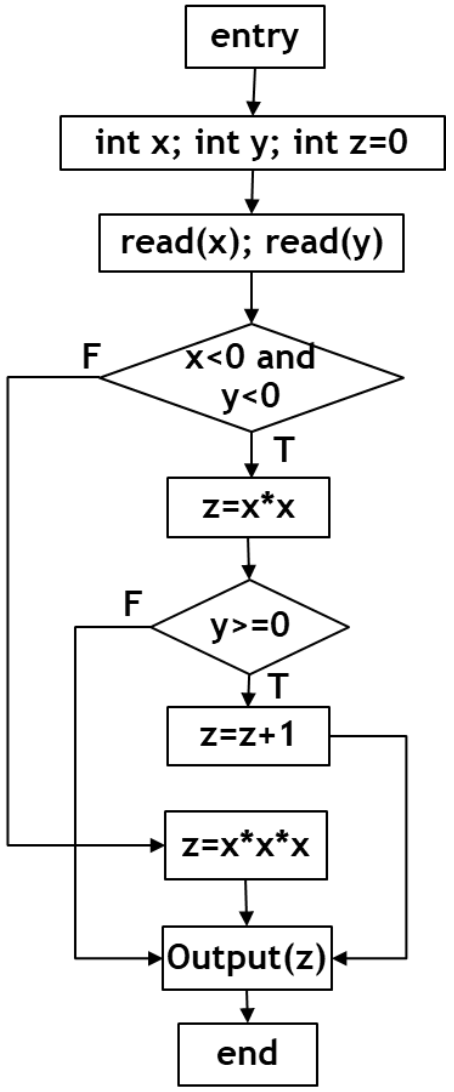
Control flow analysis

```
1:  int x; int y; int z=0;
2:  read(x); read(y);
3:  if (x<0 and y<0) {
4:      z=x*x;
5:      if (y>=0)
6:          z=z+1;
7:  }
8:  else z=x*x*x;
9:  output(z);
```



Control flow analysis

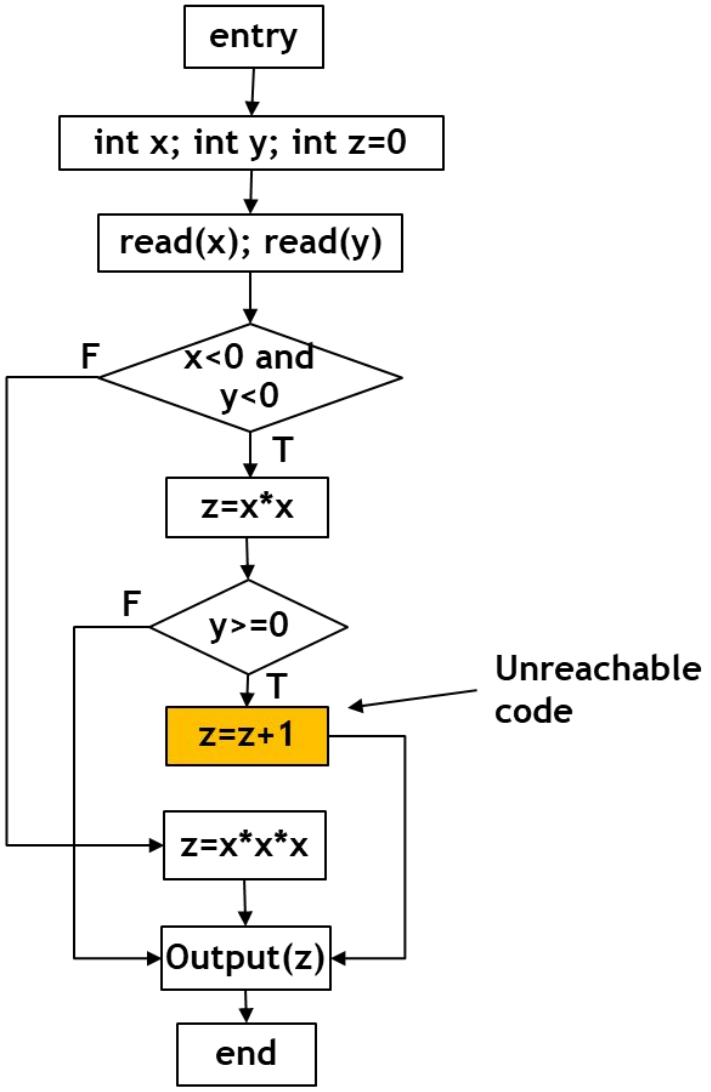
```
1:  int x; int y; int z=0;
2:  read(x); read(y);
3:  if (x<0 and y<0) {
4:      z=x*x;
5:      if (y>=0)
6:          z=z+1;
7:  }
8:  else z=x*x*x;
9:  output(z);
```



Do you see any problem with this control flow graph?

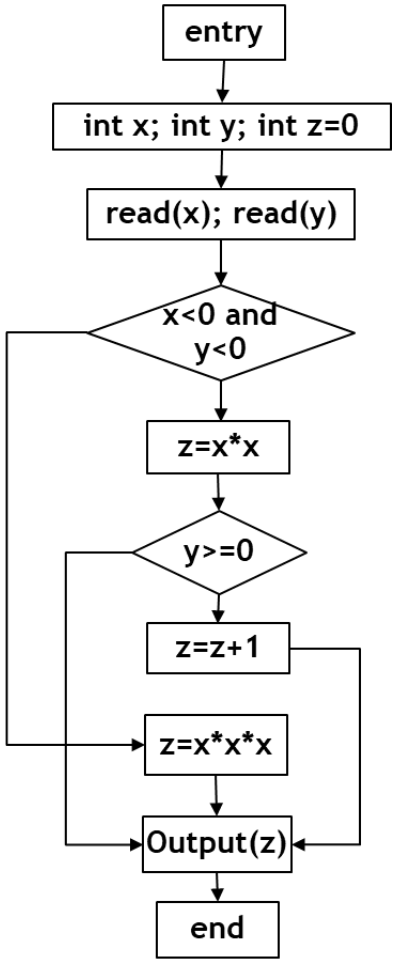
Control flow analysis

```
1:  int x; int y; int z=0;
2:  read(x); read(y);
3:  if (x<0 and y<0) {
4:      z=x*x;
5:      if (y>=0)
6:          z=z+1;
7:  }
8:  else z=x*x*x;
9:  output(z);
```



Do you see any problem with this control flow graph?

Control flow analysis - metrics



Cyclomatic complexity is a measure of the complexity of a flow graph

- (and therefore the code that the flow graph represents)

- The more complex the flow graph, the greater the measure
- In this case, it may be calculated as

  E – N + 2P

- i.e., 11 – 10 + 2 = 3

Other static metrics

Lines of code (LOC)

Operands & operators (Halstead's metrics): count operators such as "+", "-", "/", and "x", but this measure is not much used now.

Structural fan-in & fan-out: modules with high fan-in are found at the bottom of hierarchies or in libraries where they are frequently called. Modules with high fan-out are typically at the top of hierarchies, because they call to many modules (e.g., the main menu)

Nesting levels: relate to how deeply nested statements are within other IF statements.

Function calls

OO metrics: inheritence tree depth, number of methods, coupling & cohesion

DATA FLOW ANALYSIS

Data flow analysis

Examines the flow of data in a program.

It identifies the code statements where variables are defined and used.

The goal is to identify and eliminate potential anomalies that could disrupt the flow of data, leading to program malfunctions or erroneous outputs.

Data flow analysis

A program variable is DEFINED whenever its value is modified:

- left-hand side of an assignment
  - e.g., y = 17.
- input statement
  - e.g., read(y).
- call-by-reference parameter
  - e.g., update(x, &y).

A program variable is USED whenever its value is read:

- right-hand side of an assignment
  - e.g., y = x + 17.
- call-by-value parameter
  - e.g., y = sqrt(x).
- within branch's conditionals
  - e.g., if (x > 0) { ... }.

Variable used within a branch's conditionals is a **predicate-use** or "**p-use**"

Any other use is a **computation-use** or "**c-use**"

Code snippet example:

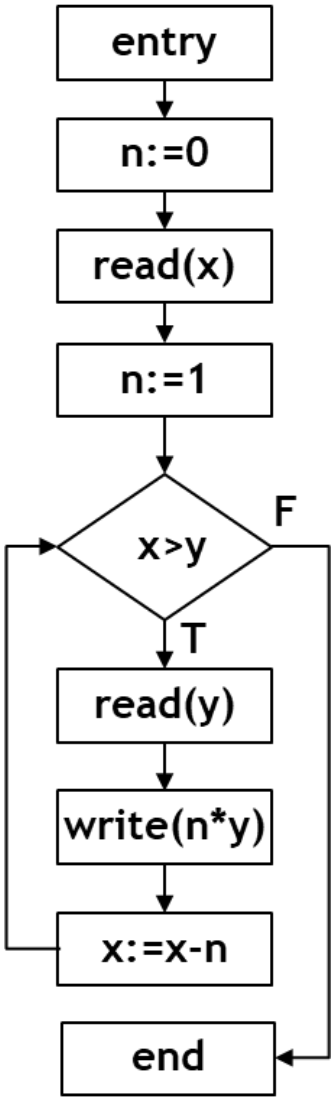
```
if (x > 0) {        -- p-use of x
    print(y);       -- c-use of y
}
```

Data flow analysis

```
1:  n=0;
2:  read(x);
3:  n := 1
4:  while (x>y) do
5:      begin
6:          read(y)
7:          write(n*y)
8:          x := x-n
9:      end
```

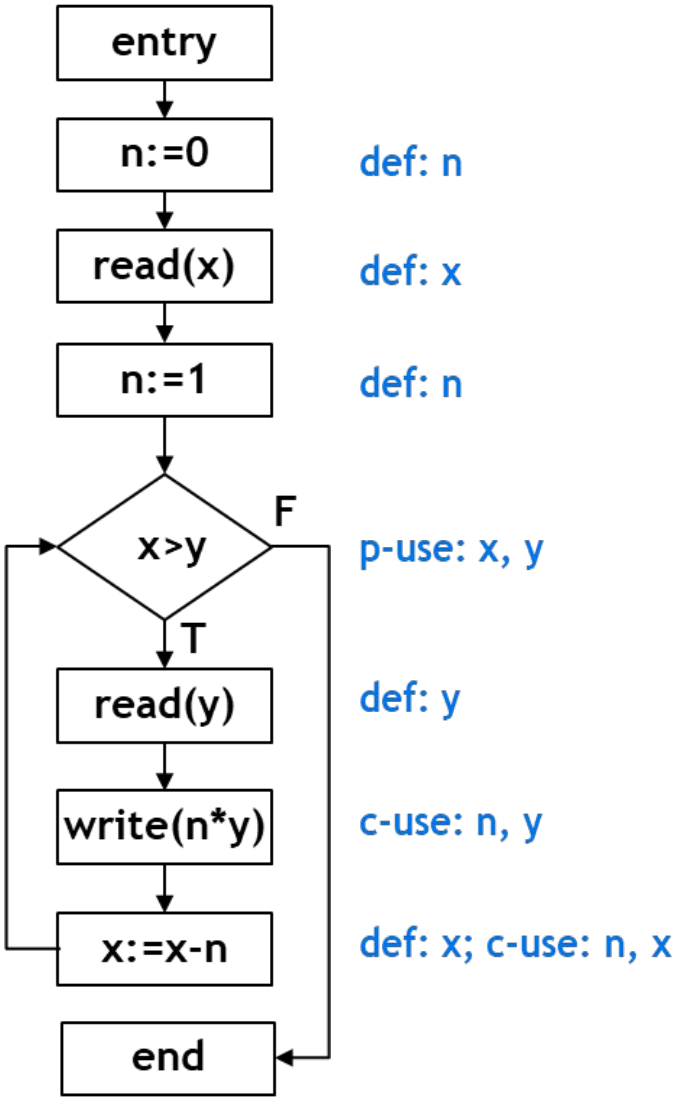
Data flow analysis

```
1:  n=0;
2:  read(x);
3:  n := 1
4:  while (x>y) do
5:      begin
6:          read(y)
7:          write(n*y)
8:          x := x-n
9:      end
```



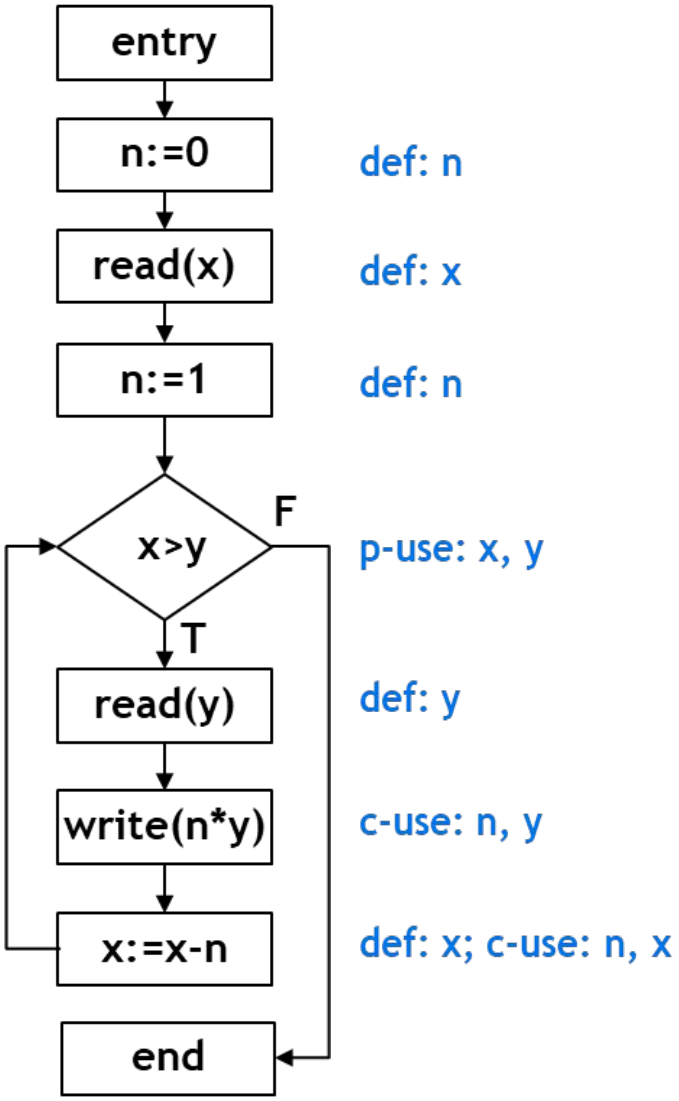
Data flow analysis

```
1:  n=0;
2:  read(x);
3:  n := 1
4:  while (x>y) do
5:      begin
6:          read(y)
7:          write(n*y)
8:          x := x-n
9:      end
```



Data flow analysis

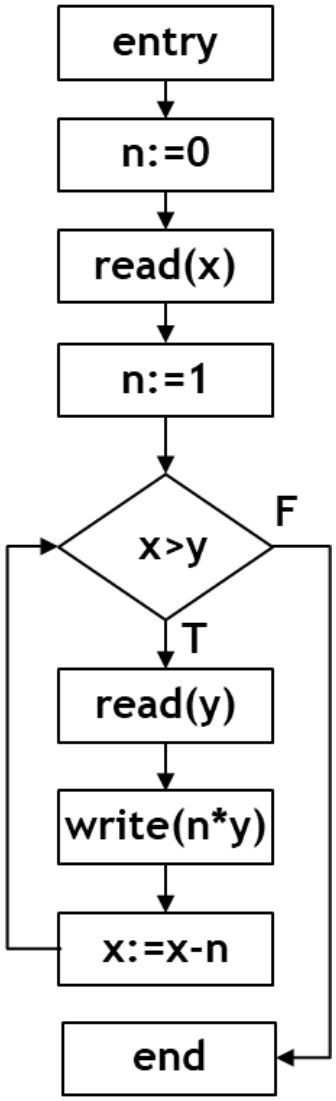
```
1:  n=0;
2:  read(x);
3:  n := 1
4:  while (x>y) do
5:      begin
6:          read(y)
7:          write(n*y)
8:          x := x-n
9:      end
```



Do you see any problem with this data flow graph?

Data flow analysis

```
1:  n=0;
2:  read(x);
3:  n := 1
4:  while (x>y) do
5:      begin
6:          read(y)
7:          write(n*y)
8:          x := x-n
9:      end
```



def: n

def: x

def: n

n is redefined without being used

p-use: x, y

y is used without being defined (in the first iteration of the loop)

def: y

c-use: n, y

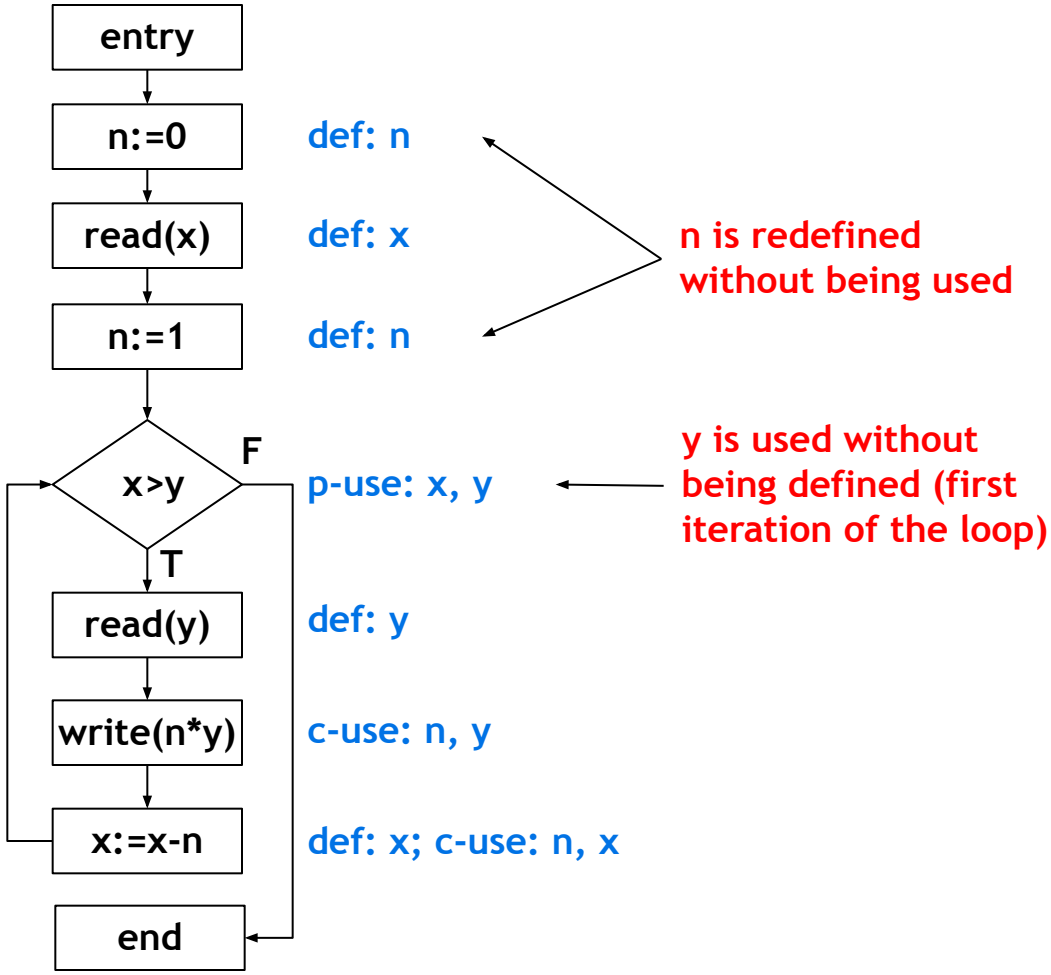
def: x; c-use: n, x

Do you see any problem with this data flow graph?

Data flow analysis

Do you see any problem with this data flow graph?

```
1:      n := 0
2:      read(x)
3:      n := 1
4:      while (x>y) do
5:        begin
6:          read(y)
7:          write(n*y)
8:          x := x-n
9:        end
```



Example 1

Static Analysis Report

Project: MyJavaApp
Analysis Date: 2023-09-24
Tool: StaticAnalyzerY

Summary:
- Total Files Analyzed: 1
- Total Issues Found: 1
- Issues by Severity:
- High: 1

```java
public class Example {
    public static void main(String[] args) {
        String str = null;
        int length = str.length();
        System.out.println("Length of str: " + length);
    }
}
```

File: Example.java

Issue 1 (High):
- Description: Null Pointer Dereference
- Location: Line 4, Column 21
- Recommendation: Check if the reference 'str' is null before invoking 'length()'.

[Generated by ChatGPT]

Example 2

Static Analysis Report

Project: MyJavaApp
Analysis Date: 2023-09-24
Tool: StaticAnalyzerX

Summary:
- Total Files Analyzed: 1
- Total Issues Found: 1
- Issues by Severity:
  - Low: 1

```java
public class Example {
    public static void main(String[] args) {
        int x = 5;
        System.out.println("Hello, world!");
    }
}
```

File: Example.java

Issue 1 (Low):
- Description: Unused Variable
- Location: Line 3, Column 9
- Recommendation: Remove or refactor the unused variable 'x'.

[Generated by ChatGPT]

Example 3

Static Analysis Report

Project: MyJavaApp
Analysis Date: 2023-09-24
Tool: StaticAnalyzerZ

Summary:
- Total Files Analyzed: 1
- Total Issues Found: 1
- Issues by Severity:
  - High: 1

```
public class BufferOverflowExample {
    public static void main(String[] args) {
        int[] numbers = new int[5];
        int index = 10;


        int value = numbers[index];

        System.out.println("Value at index " + index + ": " + value);
    }
}
```

File: BufferOverflowExample.java

Issue 1 (High):
- Description: Potential Buffer Overflow
- Location: Line 7
- Recommendation: Ensure that the array index 'index' is within the valid range before accessing the array element.

[Generated by ChatGPT]

Example 4

```java
public class CodingStyleViolationExample {
    public static void main(String[] args) {
        int x=10;
        int y = 20;
```

Static Analysis Report

Project: MyJavaApp
Analysis Date: 2023-09-24
Tool: StaticStyleChecker

Summary:
- Total Files Analyzed: 1
- Total Issues Found: 2
- Issues by Severity:
  - Low: 2

File: CodingStyleViolationExample.java

Issue 1 (Low):
  - Description: Coding Style Violation (Variable Declaration)
  - Location: Line 3
  - Recommendation: Use spaces before and after '=' in variable declarations for better readability.

[Generated by ChatGPT]

Example 5

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class SQLInjectionExample {
    public static void main(String[] args) {
        String userInput = “Ana' OR '1'='1";
        String sqlQuery = "SELECT * FROM users WHERE username = '" + userInput + "'";

        try {
            // Establish a database connection
            Connection connection = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/mydb",
                    “username", "password");

            // Create a SQL statement
            Statement statement = connection.createStatement();

            // Execute the SQL query
            ResultSet resultSet = statement.executeQuery(sqlQuery);

            // Process the query results...
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Static Analysis Report

Project: MyJavaApp
Analysis Date: 2023-09-24
Tool: StaticAnalyzerX

Summary:
- Total Files Analyzed: 1
- Total Issues Found: 1
- Issues by Severity:
  - High: 1

File: SQLInjectionExample.java

Issue 1 (High):
- Description: Security Vulnerability (SQL Injection)
- Location: Line 8
- Recommendation: Use parameterized queries or prepared statements to construct SQL queries with user input to prevent SQL injection attacks.

[Generated by ChatGPT]

Example 5

```java
public class DeadCodeExample {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;

        if (x > 10) {
            System.out.println("                                    ");
        }

        int z = x + y;
        System.out.println("The sum is: " + z);
    }
}
```

Static Analysis Report

Project: MyJavaApp
Analysis Date: 2023-09-24
Tool: StaticAnalyzerX

Summary:
- Total Files Analyzed: 1
- Total Issues Found: 1
- Issues by Severity:
  - Low: 1

File: DeadCodeExample.java

Issue 1 (Low):
- Description: Dead Code
- Location: Line 7
- Recommendation: Remove the dead code block to improve code readability and maintainability.

[Generated by ChatGPT]

Example 6

```java
public class UserDetails {
    public void getUserDetails(String username) {
        try (
            Connection connection = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/users_db",
                "root",
                "password"
            );
            Statement statement = connection.createStatement()
        ) {

            // Vulnerable SQL query (direct concatenation of user input)
            String query = "SELECT * FROM users WHERE username = '" + username + "'";
            ResultSet result = statement.executeQuery(query);

            // Process result
            while (result.next()) {
                System.out.println("User: " + result.getString("username"));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

[Generated by ChatGPT]

Example 6

```java
public class UserDetails {
    public void getUserDetails(String username) {
        try (
            Connection connection = DriverManager.get
                "jdbc:mysql://localhost:3306/users_db
                "root",
                "password"
            );
            Statement statement = connection.createSt
        ) {

            // Vulnerable SQL query (direct concatena
            String query = "SELECT * FROM users WHERE
            ResultSet result = statement.executeQuery

            // Process result
            while (result.next()) {
                System.out.println("User: " + result.
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Report Example

Static Testing Tool: SonarQube

Analysis Date: 2024-12-04

Project Name: User Management System

File: `UserDetails.java`

Line Number: 14

Issue Detected: SQL Injection Vulnerability

- **Severity:** High
- **Description:** User input is directly concatenated into an SQL query string. This allows attackers to execute arbitrary SQL commands by injecting malicious inputs.
- **Issue Details:**
  - **Input Source:** `username` (method parameter)
  - **Vulnerable Line:** `String query = "SELECT * FROM users WHERE username = '" + username + "'";`
  - **Risk:** An attacker could supply input such as `' OR 1=1 --` to bypass authentication or execute unauthorized queries.

Recommendation:

- Use **PreparedStatement** to avoid SQL injection vulnerabilities.

[Generated by ChatGPT]